7.1 A fresh baseline and a failed promise

We begin with a promise: a Dessert object never stores a negative calorie count. If a caller asks it to break that promise, the object throws an exception before storing the bad value. Today we will deliberately leave that exception uncaught to see what it does.

Set up this week's files

Use the existing student repository, PHP 8.4 or newer, existing Composer autoloader, and existing App PSR-4 mapping. Do not create composer.json, change autoload mappings, or install Twig. All paths below belong to the student repository. No archived files need to be copied.

Create `public/tutorials/class7/` and `src/Tutorials/Class7/`. PHP files use strict types and no closing PHP tag. The namespace corresponds to the path under src/.

Create the model

Create `src/Tutorials/Class7/Food.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

class Food
{
    public function __construct(protected string $name)
    {
    }

    public function getName(): string
    {
        return $this->name;
    }

    public function __toString(): string
    {
        return "Food: {$this->name}";
    }
}
```

The promoted protected name property is available to Dessert. A public getter will later allow Twig to read it. The string conversion gives our first examples readable output.

Create `src/Tutorials/Class7/Dessert.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

use Exception;

class Dessert extends Food
{
    private int $calories;

    public function __construct(string $name, int $calories)
    {
        parent::__construct($name);
        $this->setCalories($calories);
    }

    public function getCalories(): int
    {
        return $this->calories;
    }

    public function setCalories(int $calories): void
    {
        if ($calories < 0) {
            throw new Exception('Calories cannot be negative.');
        }

        $this->calories = $calories;
    }

    public function __toString(): string
    {
        return "Dessert: {$this->name}, {$this->calories} calories";
    }
}
```

The constructor uses the setter so construction and later changes follow the same rule. The assignment comes after validation. We import PHP's Exception class from the global namespace; without an import or leading backslash, PHP would look for Exception in our own namespace.

Deliberately leave a failure uncaught

Create `public/tutorials/class7/index.php`:

```php
<?php
declare(strict_types=1);

use App\Tutorials\Class7\Dessert;
use App\Tutorials\Class7\Food;
```

```
require_once __DIR__ . '/../../../vendor/autoload.php';

echo new Food('Apple'), '<br>', PHP_EOL;
echo 'Before construction', '<br>', PHP_EOL;
$dessert = new Dessert('Berry crumble', -1);
echo $dessert, '<br>', PHP_EOL;
echo 'After construction', '<br>', PHP_EOL;
```

Run this controlled demonstration locally from the student repository root:

```
php public/tutorials/class7/index.php
```

Expect Food: Apple and Before construction, followed by an uncaught Exception and a stack trace. Neither the dessert description nor After construction appears. The exception interrupts the setter, constructor, and caller. `$dessert` was not assigned a new object because the right-hand side failed.

Read the trace from the throw site through the setter call in the constructor and the constructor call in index.php. It describes how execution reached the problem. The exact paths and line numbers depend on your repository.

This is a CLI diagnostic, not a visitor-facing error page. Keep the intentional uncaught failure local. We will replace it with controlled responses in subsequent lessons.

Compare success

Change only the constructor argument from -1 to 500 and run again. All four output lines appear, including Dessert: Berry crumble, 500 calories. Zero is also valid in our calorie model. Restore -1 before 7.2 so we can compare the same failure with a catch.